

Logical Agents and Propositional Logic

CHAPTER 7 AND 8



In search-based agents, the search algorithm depends on domain-specific state representations, successor functions, and goal tests, so it must be modified for each new domain, making it brittle and non-reusable.

Moving on...

❑ Atomic search space

- Brittle, had to be modified for every domain
- Enumerating all possible states – impossible for large problems
- Child generator functions, goal test functions are black box to the search algorithm

Atomic search space

States are treated as indivisible “atoms”

No internal structure or meaning is used

Brittle systems

The search algorithm must be modified for each new domain

Not reusable or flexible

State space explosion

Enumerating all possible states is impossible for large problems

Black-box functions

Child generator and goal test are hidden from the algorithm

The agent has no understanding of why a state is good or bad

Knowledge Based Agents

□ Two main components

- Knowledge Base

- Set of sentences in a formal language
- Represents some assertion about the world

- Inference Engine

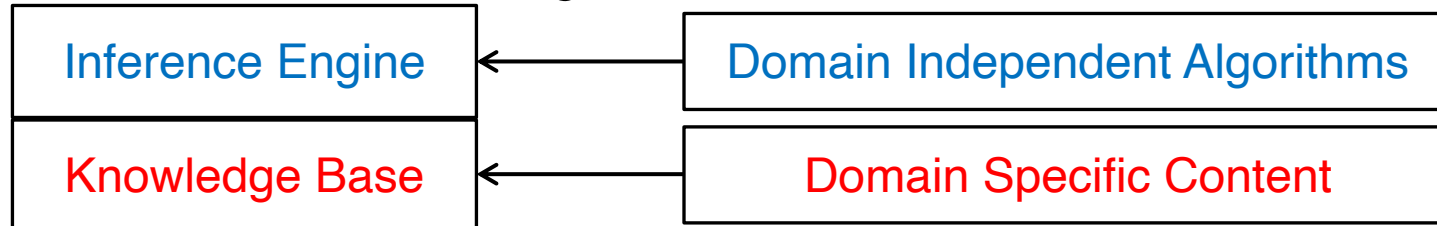
- Mechanism to derive new sentences from old

□ Declarative Approach

- TELL the agent what it needs to know

- ASK itself what to do?

Separation of knowledge and reasoning is a major advantage.



Knowledge Based Agent

High-level flow:

- The agent receives a percept
- Converts percept into a logical sentence
- TELLs the KB about the percept
- ASKs the KB what action to take
- Records the chosen action
- Executes the action

function **KB-AGENT**(*percept*) **returns** an *action*

static: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, **MAKE-PERCEPT-SENTENCE**(*percept*, *t*))

action $\leftarrow$ **ASK**(*KB*, **MAKE-ACTION-QUERY**(*t*))

TELL(*KB*, **MAKE-ACTION-SENTENCE**(*action*, *t*))

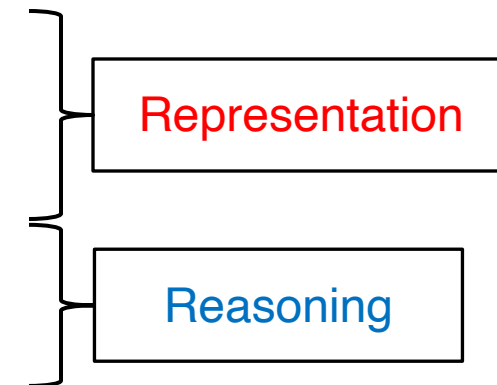
t $\leftarrow$ *t* + 1

return *action*

Intelligence comes from combining representation + reasoning.

□ The agent must

- Represent states and actions
- Incorporate new percepts
- Update internal representations of the world
- Deduce hidden properties of the world
- Deduce appropriate actions



Wumpus World - Example

Performance Measure

- +1000 for gold
- 1000 for death
- 1 per step
- 10 for shooting arrow

Environment Properties

- Adjacent to Wumpus → Stench
- Adjacent to Pit → Breeze
- Same square as gold → Glitter

Sensors

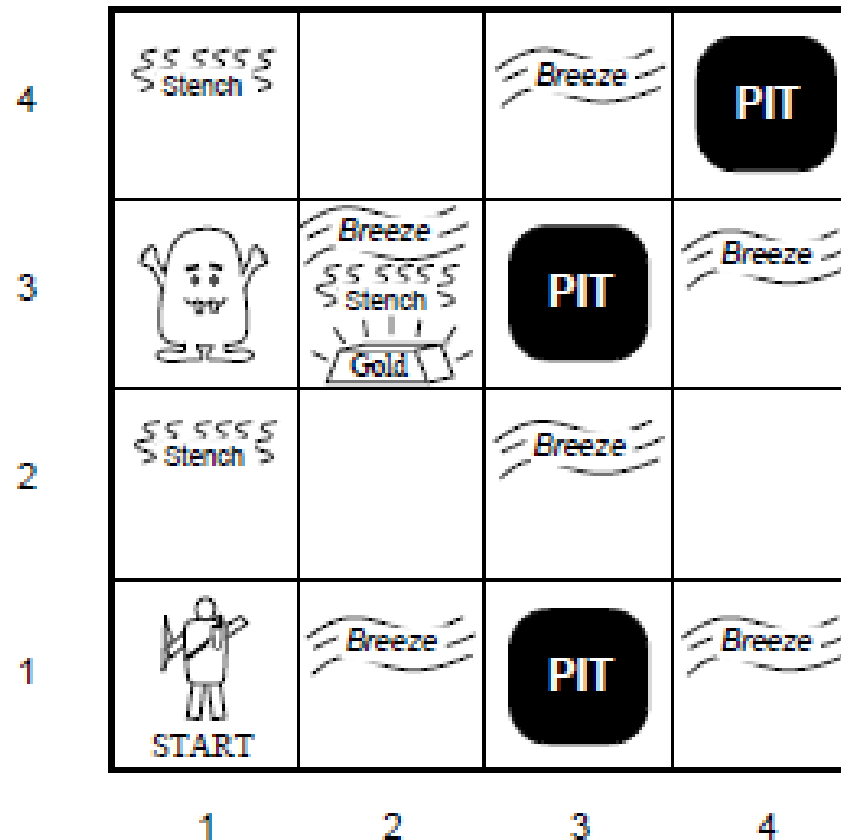
Stench, Breeze, Glitter, Bump, Scream

Actuators

TurnLeft, TurnRight, Forward, Grab, Shoot, Climb

Environment Classification

- Partially observable
- Deterministic
- Sequential
- Static
- Discrete
- Single-agent



Climb is a special action: it allows the agent to leave the cave entirely, ending the episode.

WW PEAS Description and Characterization

❑ Performance measure

- gold +1000 death -1000
- -1 per step, -10 for arrow

❑ Environment

- Squares adjacent to Wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter iff gold is in the same square
- Shooting kills Wumpus if the agent is facing it
- Shooting uses up the only arrow
- Grabbing picks up the gold if the agent is in the same square
- Climbing in [1,1] gets the agent out of the cave

"The only way to exit the Wumpus World is to climb out at the starting square [1,1]."

❑ Sensors

- Stench, Breeze, Glitter, Bump, Scream

❑ Actuators

- TurnLeft, TurnRight, Forward, Grab, Shoot, Climb

❑ Observability: No – only local perception

❑ Deterministic: Yes, outcome is exactly specified

❑ Episodic: No – sequential at the level of actions

❑ Static: Yes, Wumpus, pits and gold do not move

❑ Discrete: Yes

❑ Single Agent: Yes, Wumpus is a natural feature of the environment

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
B = Breeze
G = Glitter/Gold
OK = Safe Square
P = Pit
S = Stench
V = Visited
W = Wumpus

Knowledge Base:

- [1,2] and [2,1] are safe

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V → OK	2,1 A OK B	3,1 P?	4,1

A = Agent
B = Breeze
G = Glitter/Gold
OK = Safe Square
P = Pit
S = Stench
V = Visited
W = Wumpus

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V A ← V OK	2,1 ← V OK B	3,1 P?	4,1

A = Agent
B = Breeze
G = Glitter/Gold
OK = Safe Square
P = Pit
S = Stench
V = Visited
W = Wumpus

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter/Gold
OK = Safe Square
P = Pit
S = Stench
V = Visited
W = Wumpus

- [1,2] is smelly
 - Wumpus in [1,3] or [2,2] or [1,1]
- [1,2] is not breezy
 - Therefore no pit in [2,2]
- [2,1] is not smelly
 - Therefore no Wumpus in [2,2]
- [2,2] has not pit or Wumpus
 - Therefore [2,2] is safe
- [1,2] is smelly and [2,2] and [1,1] are safe
 - Therefore Wumpus is in [1,3]

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

A = Agent
 B = Breeze
 G = Glitter/Gold
 OK = Safe Square
 P = Pit
 S = Stench
 V = Visited
 W = Wumpus

- [1,2] is smelly
 - Wumpus in [1,3] or [2,2] or [1,1]
- [1,2] is not breezy
 - Therefore no pit in [2,2]
- [2,1] is not smelly
 - Therefore no Wumpus in [2,2]
- [2,2] has not pit or Wumpus
 - Therefore [2,2] is safe
- [1,2] is smelly and [2,2] and [1,1] are safe
 - Therefore Wumpus is in [1,3]

Exploring the WW

1,4	2,4 P?	3,4	4,4
1,3 W	2,3 OK A B S G	3,3 P?	4,3
1,2 V OK S	2,2 V OK	3,2 OK	4,2
1,1 V OK	2,1 V OK B	3,1 P?	4,1

A = Agent
 B = Breeze
 G = Glitter/Gold
 OK = Safe Square
 P = Pit
 S = Stench
 V = Visited
 W = Wumpus

- [1,2] is smelly
 - Wumpus in [1,3] or [2,2] or [1,1]
- [1,2] is not breezy
 - Therefore no pit in [2,2]
- [2,1] is not smelly
 - Therefore no Wumpus in [2,2]
- [2,2] has not pit or Wumpus
 - Therefore [2,2] is safe
- [1,2] is smelly and [2,2] and [1,1] are safe
 - Therefore Wumpus is in [1,3]

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

Exploring the WW

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter/Gold
OK = Safe Square
P? = Pit?

How do we represent rules of the world and the percepts encountered?

Knowledge Base:

- [1,2] and [2,1] are safe
- [2,1] is breezy
 - Pit in [3,1] or [2,2]?

- Wumpus in [1,3] or [2,2] or [1,1]
- [1,2] is not breezy
 - Therefore no pit in [2,2]
- [2,1] is not smelly
 - Therefore no Wumpus in [2,2]
- [2,2] has not pit or Wumpus
 - Therefore [2,2] is safe
- [1,2] is smelly and [2,2] and [1,1] are safe
 - Therefore Wumpus is in [1,3]

Logical Representation

- ❑ Logic - formal languages for representing information such that conclusions can be drawn
- ❑ Sentence – individual piece of knowledge
- ❑ Syntax – What expressions are legal?
 - In the language of arithmetic $x + 2 \geq y$ is a sentence, but $x^2 + y \geq$ is not a sentence
- ❑ Semantics – What the legal expressions mean?
 - $x + 2 \geq y$ is true iff the number $x + 2$ is no less than the number y
 - $x + 2 \geq y$ is true in a world where $x = 5, y = 4$
 - $x + 2 \geq y$ is false in a world where $x = 3, y = 7$

Models

- ❑ Formally structured worlds with respect to which truth can be evaluated
- ❑ We say m is a model of a sentence α if α is true in m
- ❑ $M(\alpha)$ is the set of all models of α
- ❑ Example: $x + 2 \geq y$ is true in a world
 - where $x = 5, y = 4$
 - where $x = 4, y = 4$
 - where $x = 4, y = 6$
 - where $x = 5, y = 3$

What is a model?

A model is a formally defined world where sentences can be evaluated as true or false

A model m satisfies sentence α if α is true in m

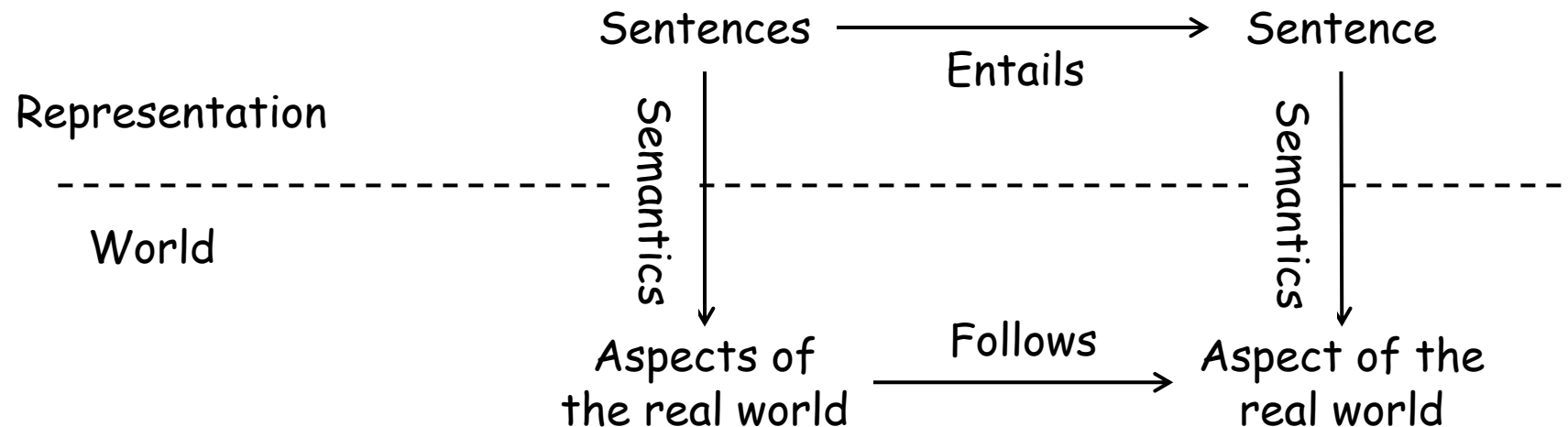
Entailment

□ Means that one thing follows from another:

$$KB \models \alpha$$

□ Knowledge Base KB entails a sentence α iff α is true in all worlds where KB is true

□ Relationship between sentences that is based on semantics



Entailment

- KB:
 - Sun rises in the East
 - Arunachal Pradesh is the Eastern most state of India
- Entails
 - Sun rises first in Arunachal Pradesh in India

- KB:
 - $x + y = 4$
 - $x = 2$
- Entails
 - $4 = x + y$
 - $y = 2$

- KB:
 - Learning is maximum in fun classes
 - AI class is a fun class
- Entails
 - Learning is maximum in AI class

□ KB entails a sentence α iff α is true in all the worlds where KB is also true

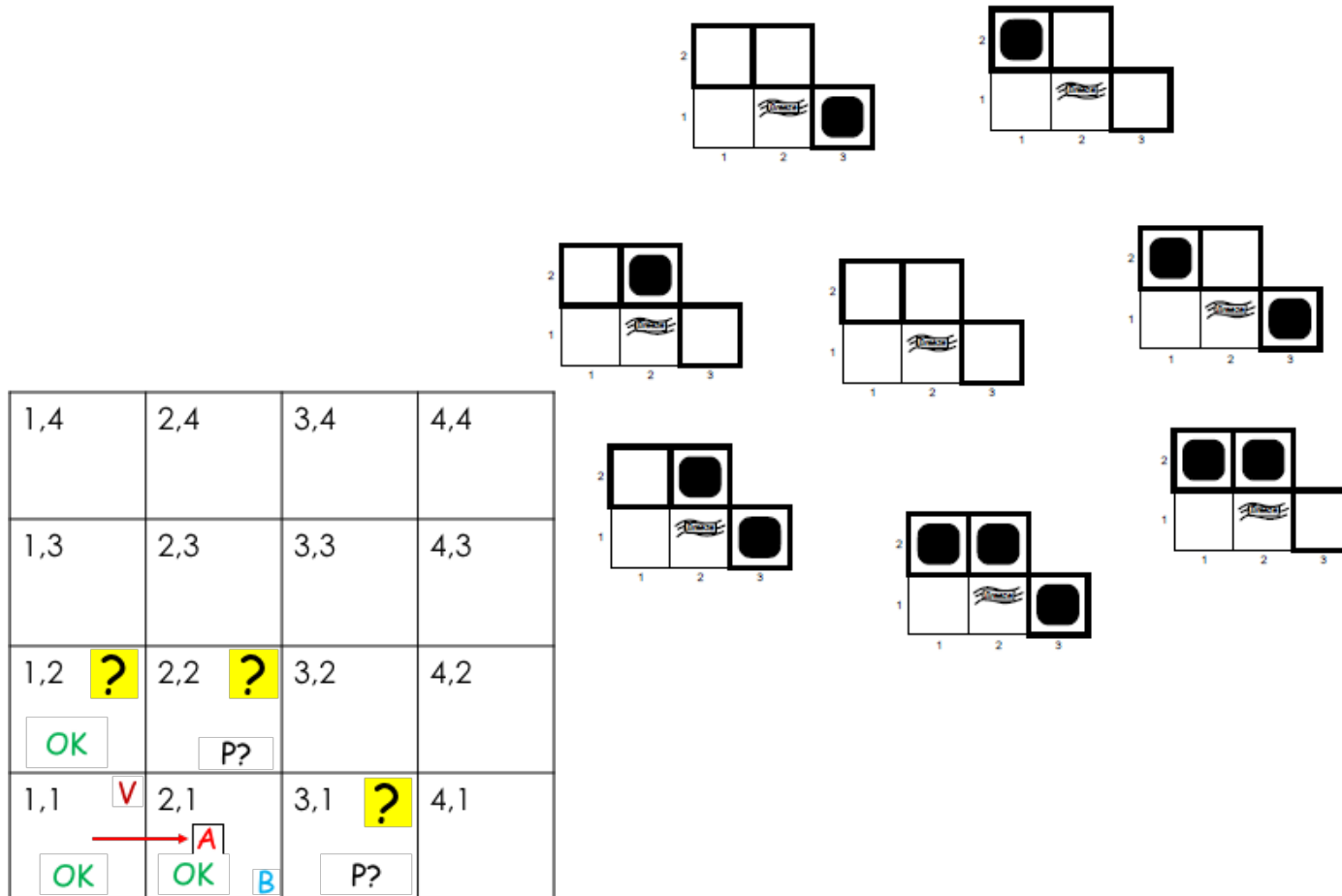
□ $KB \models \alpha$ iff $M(KB) \subseteq M(\alpha)$

Entailment in the WW

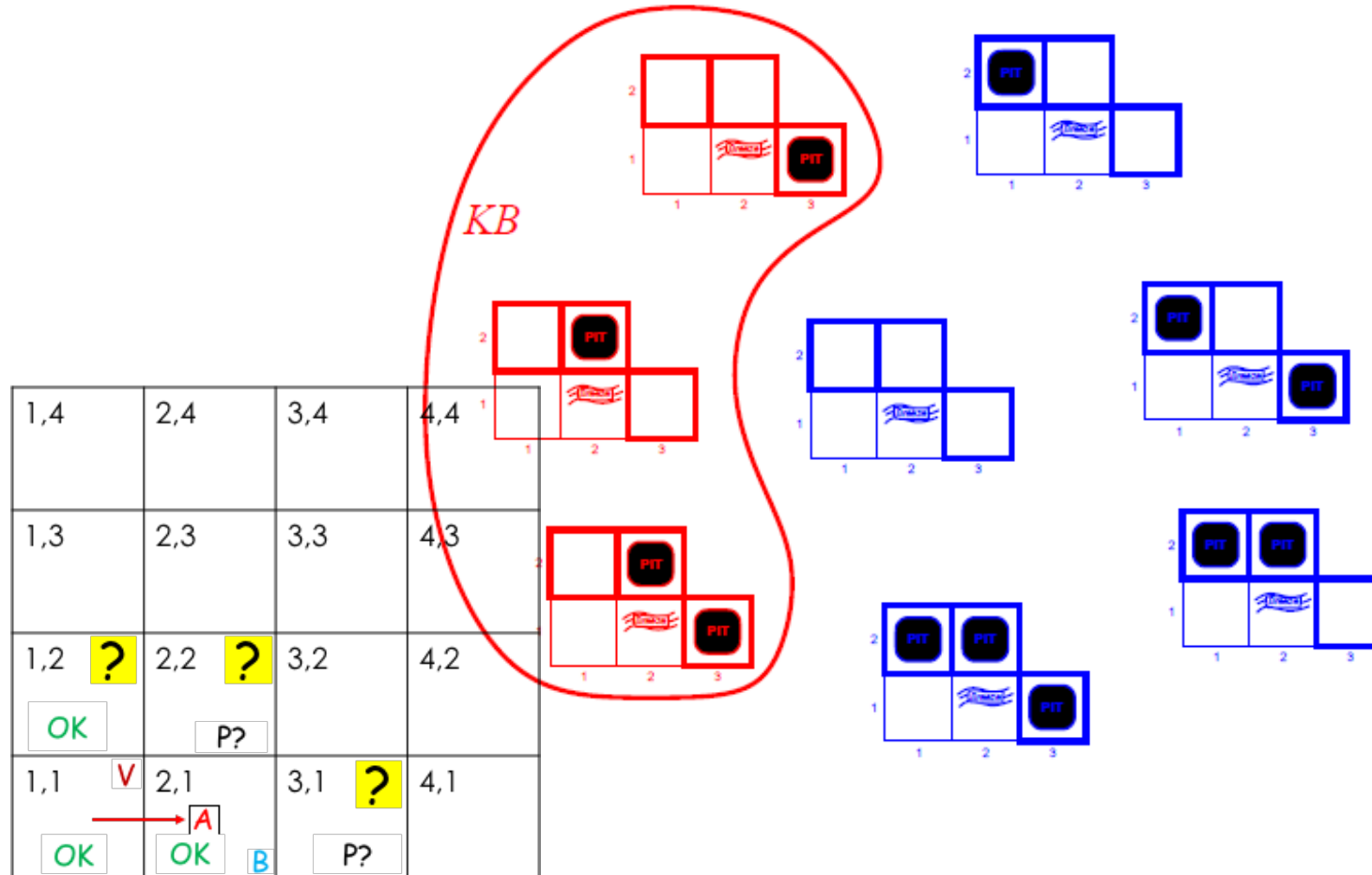
1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 ?	2,2 ? p?	3,2	4,2
1,1 V OK	2,1 A OK B	3,1 ? p?	4,1

- Situation after detecting nothing in [1, 1], moving right, breeze in [2, 1]
- Consider possible models for ?s assuming only pits
- 3 Boolean models = 8 possible models

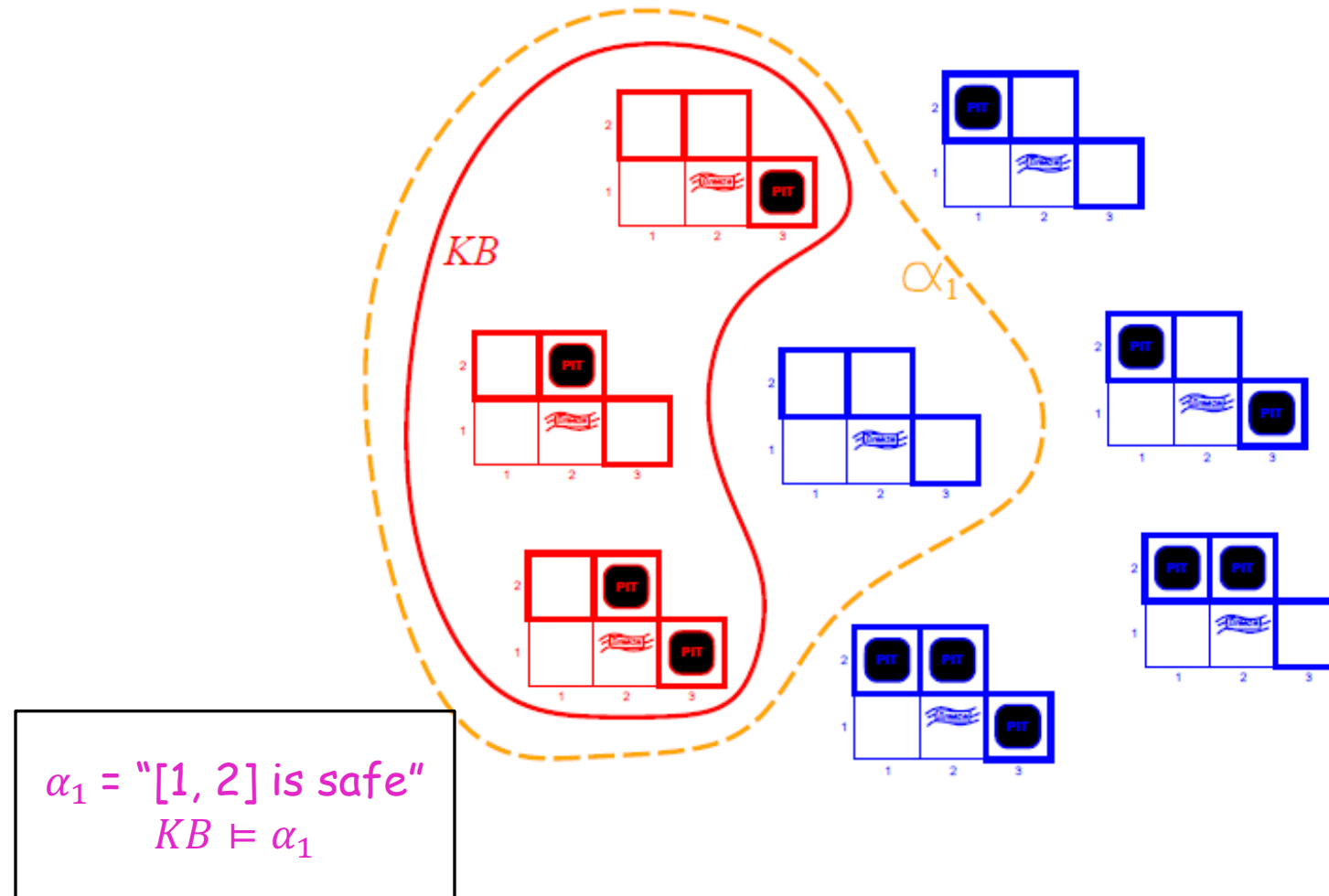
8 possible models



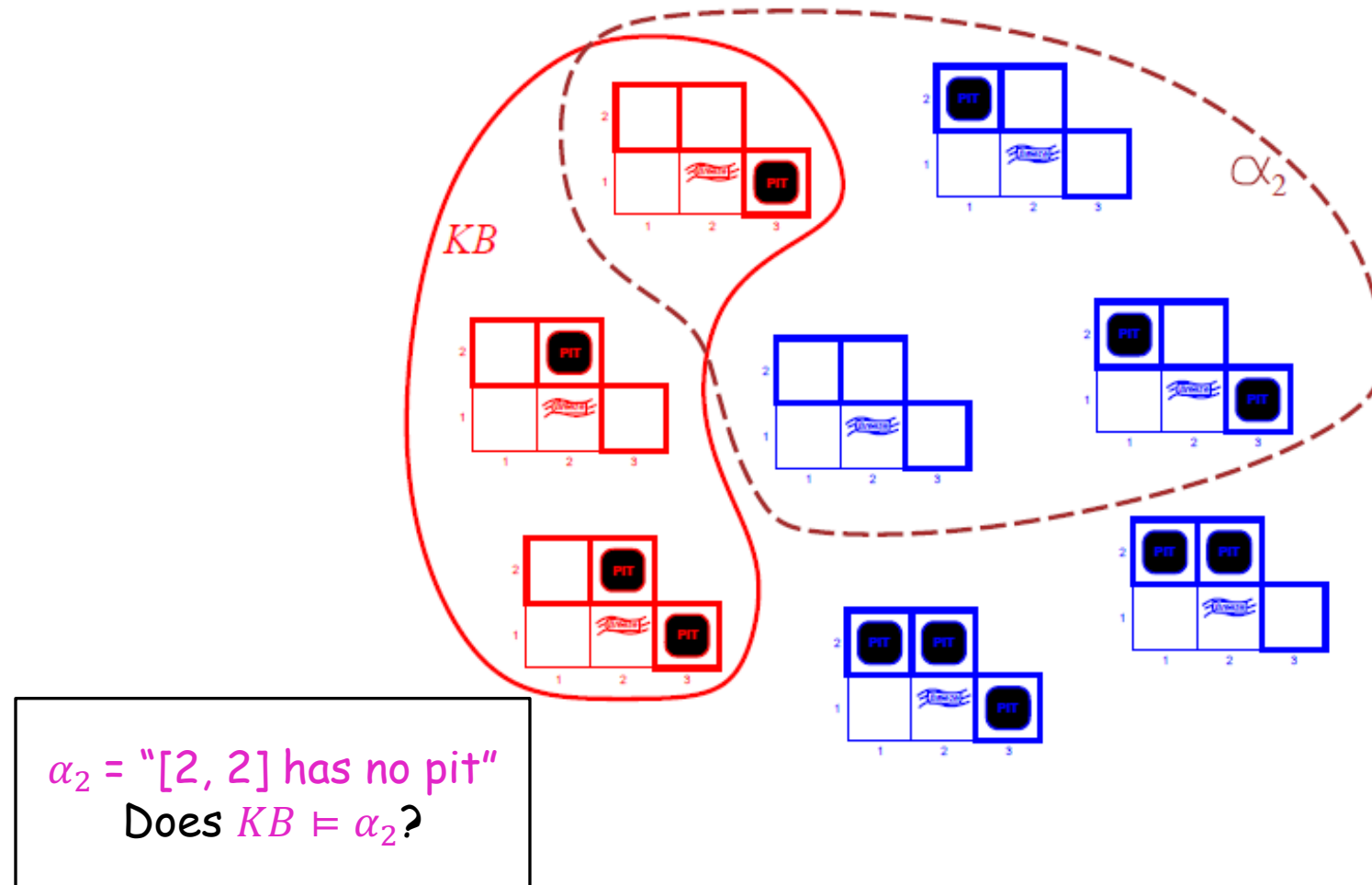
WW Models - KB



WW models – Entailment by model checking



WW models - Entailment



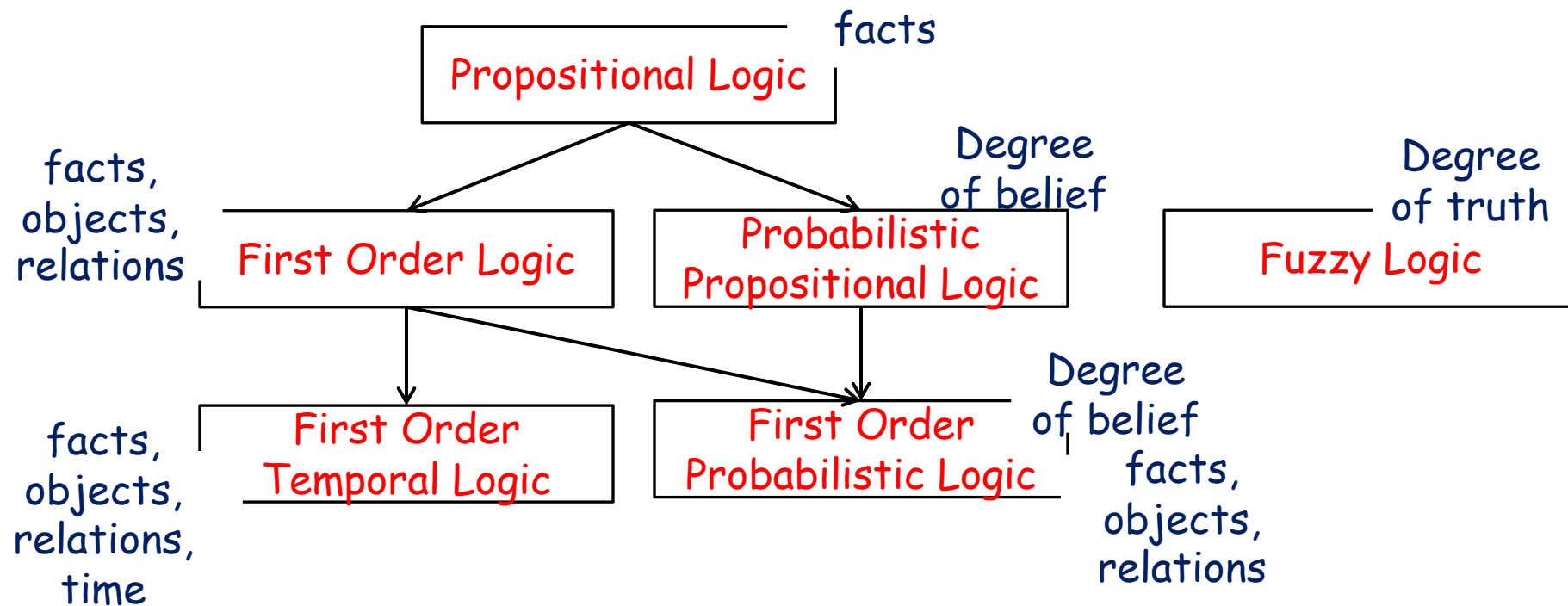
Inference

- ❑ $KB \vdash_i \alpha$ – sentence α can be derived from KB by procedure i .
- ❑ All consequences of KB – haystack
- ❑ α is the needle
- ❑ Entailment = needle in the haystack
- ❑ Inference = Finding the needle
- ❑ **Sound**: If the inference algorithm only derives entailed sentences
- ❑ **Complete**: If the algorithm can derive any entailed sentence



Types of Logic

- ❑ Ontological commitments: What exists? –facts, objects, relations, time?
- ❑ Epistemological commitments: What states of knowledge?



Propositional Logic - Syntax

- ❑ Simplest Logic – illustrates the basic ideas
- ❑ Atomic sentences – single proposition symbols
 - Can either be *True* or *False* (meaning is fixed)
 - E.g. *A*, *B* that can be either true or false
 - *P*₁₂ or *P*₂₂ etc. used to denote properties of the world
 - *P*₁₂ - pit in square [1,2] – can be true or false in the WW
- ❑ Complex sentences are constructed from simpler sentences recursively using logical connectives
 - Negation - $\neg S$
 - literal
 - Conjunction - $S_1 \wedge S_2$
 - Disjunction - $S_1 \vee S_2$
 - Implication - $S_1 \Rightarrow S_2$
 - Premise (antecedent)
 - Conclusion (consequent)
 - Biconditional - $S_1 \Leftrightarrow S_2$

Propositional Logic – Syntax Backus-Naur Form

$Sentence \rightarrow AtomicSentence \mid ComplexSentence$

$AtomicSentence \rightarrow True \mid False \mid P \mid Q \mid R \mid \dots$

$ComplexSentence \rightarrow (Sentence) \mid [Sentence]$

$\mid \neg Sentence$

$\mid Sentence \wedge Sentence$

$\mid Sentence \vee Sentence$

$\mid Sentence \Rightarrow Sentence$

$\mid Sentence \Leftrightarrow Sentence$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Propositional Logic - Semantics

- A model specifies *true/false* for each proposition symbol.
- Rules for evaluating truth w.r.t. a model *m*:

$\neg S$	is true iff	S	is false		
$S_1 \wedge S_2$	is true iff	S_1	is true and	S_2	is true
$S_1 \vee S_2$	is true iff	S_1	is true or	S_2	is true
$S_1 \Rightarrow S_2$	is true iff	S_1	is false or	S_2	is true
	i.e., is false iff	S_1	is true and	S_2	is false
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$	is true and	$S_2 \Rightarrow S_1$	is true

P_{12}	P_{22}	P_{31}	$\neg P_{12} \wedge (P_{22} \vee P_{31})$
<i>false</i>	<i>false</i>	<i>true</i>	$= \neg \text{false} \wedge (\text{false} \vee \text{true})$
			$= \text{true} \wedge \text{true} = \text{true}$

Truth Tables for connectives

S_1	S_2	$\neg S_1$	$S_1 \wedge S_2$	$S_1 \vee S_2$	$S_1 \Rightarrow S_2$	$S_1 \Leftrightarrow S_2$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true



Psst... here's a useful tip:
 $S_1 \Rightarrow S_2$ is equivalent to $\neg S_1 \vee S_2$
Prove it!

Exercise: Write an algorithm for computing the truth value of a propositional logic sentence s given a model m

Wumpuspositional Logic

□ Syntax for WW sentences

- P_{ij} is true if there is a pit in $[i, j]$
- B_{ij} is true if there is breeze in $[i, j]$

□ Knowledge Base:

- True Statements

$$\neg P_{11}, \neg B_{11}, B_{21}$$

- Properties of the Wumpus World

- “Pits cause breeze in adjacent squares”

$$B_{11} \Leftrightarrow (P_{21} \vee P_{12})$$

$$B_{21} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{31})$$

- A square is breezy iff there is an adjacent pit



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 OK	2,1 V → A OK B	3,1 P?	4,1

Inference Proof/Techniques

□ Model Checking

- Truth Table enumeration (exponential in n)
- Efficient Backtracking algorithms
 - e.g. David-Putnam-Logemann-Loveland (DPLL)
- Local Search Algorithms (sound but incomplete)
 - e.g. randomized hill climbing (WalkSAT)

□ Application of inference rules

- Generate new sentences from old in a sound way
- Proof – a sequence of inference applications
- Use inference rules as actions/operators in a standard search algorithm
- Typically require translation of sentence into a normal form.

Inference by Truth Table Enumeration

	B_{11}	B_{21}	P_{11}	P_{12}	P_{21}	P_{22}	P_{31}	KB	$\neg P_{12}$
Model 1	false	false	false	false	false	false	false	false	true
Model 2	false	false	false	false	false	false	true	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	false	true	false	false	false	false	false	false	true
	false	true	false	false	false	false	true	true	true
	false	true	false	false	false	true	false	true	true
	false	true	false	false	false	true	true	true	true
	false	true	false	false	true	false	false	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	true	true	true	true	true	true	true	false	false

In all models in which KB is true, $\neg P_{12}$ is also true.

Therefore $KB \models \neg P_{12}$

$$\neg P_{11}, \neg B_{11}, B_{21}$$

$$B_{11} \Leftrightarrow (P_{21} \vee P_{12})$$

$$B_{21} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{31})$$

Inference by Truth Table Enumeration

B_{11}	B_{21}	P_{11}	P_{12}	P_{21}	P_{22}	P_{31}	KB	P_{22}
false	false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false	false
false	true	false	false	false	false	true	true	false
false	true	false	false	false	true	false	true	true
false	true	false	false	false	true	true	true	true
false	true	false	false	true	false	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true

P_{22} is false in a model in which KB is true

Therefore, $KB \not\models P_{22}$

$$\begin{aligned}
 &\neg P_{11}, \neg B_{11}, B_{21} \\
 &B_{11} \Leftrightarrow (P_{21} \vee P_{12}) \\
 &B_{21} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{31})
 \end{aligned}$$

Inference by Enumeration

function **TT-ENTAILS?**(KB, α) **returns** *true* or *false*
 inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic
 $symbols \leftarrow$ a list of the proposition symbols in KB and α
 return **TT-CHECK-ALL**($KB, \alpha, symbols, []$)

function **TT-CHECK-ALL**($KB, \alpha, symbols, model$) **returns** *true* or *false*
 if **EMPTY?**($symbols$) **then**
 if **PL-TRUE?**($KB, model$) **then** **return** **PL-TRUE?**($\alpha, model$)
 else **return** *true*
 else do
 $P \leftarrow$ **FIRST**($symbols$); $rest \leftarrow$ **REST**($symbols$)
 return **TT-CHECK-ALL**($KB, \alpha, rest, \text{EXTEND}(P, \text{true}, model)$) **and**
 TT-CHECK-ALL($KB, \alpha, rest, \text{EXTEND}(P, \text{false}, model)$)

- Depth first exploration
- Complexity - $O(2^n)$ n – symbols

- Sound - Yes
- Complete - Yes

Propositional Theorem Proving - Logical Equivalence Laws

□ Two sentences are logically equivalent iff true in same models: $\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \text{ commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \text{ commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \text{ associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \text{ associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \text{ double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \text{ contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \text{ implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \text{ biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \text{ de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \text{ de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \text{ distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \text{ distributivity of } \vee \text{ over } \wedge$$

Propositional Theorem Proving - Validity and Satisfiability

□ A sentence is valid if it is true in **all** models (tautology)

○ $True, \alpha \vee \neg\alpha, \alpha \Rightarrow \alpha, (\alpha \wedge (\alpha \Rightarrow \beta)) \Rightarrow \beta$

□ Validity is connected to inference via **Deduction Theorem**

○ $KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid **Prove it!**

□ A sentence is satisfiable if it is true in some model

○ $\alpha \vee \beta, \gamma$ - Satisfiability (SAT) Problem – NP-Complete

□ A sentence is unsatisfiable if it is true in no model

○ $\alpha \wedge \neg\alpha$

□ Satisfiability is connected to inference via: $KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable (proof by contradiction)

Inference Technique I - Resolution

□ Motivation

$$\frac{P_{13} \vee P_{22} \quad \neg P_{22}}{P_{13}}$$

□ More generally

$$\frac{l_1 \vee \cdots \vee l_k \quad \neg l_i}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k}$$

□ Terminology – unit resolution

- Literal – proposition symbol or its negation, e.g., $\alpha, \neg\beta$
- Clause – disjunction of literals, e.g., $(\beta \vee \neg\alpha \vee \gamma)$
- Resolution assumes sentences are in **conjunctive normal form (CNF)** – conjunction of clauses, e.g., $(\alpha \vee \beta) \wedge (\gamma \vee \neg\alpha)$

Conversion to CNF

- ❑ Every sentence in Propositional logic is logically equivalent to a conjunction of clauses!
- ❑ Apply logical equivalence laws successively
- ❑ e.g. $B_{11} \Leftrightarrow (P_{12} \vee P_{21})$
 1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
 2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $(\neg \alpha \vee \beta)$
 3. Move $\neg$ inwards using de Morgan's rules and double negation
 4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

Inference Technique I - Resolution

Full Resolution rule (for CNF)

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

l_i and m_j are complimentary literals i.e. $l_i = \neg m_j$

Factoring

$$\frac{P_{11} \vee P_{22} \vee P_{31}, \quad \neg P_{11} \vee P_{22}}{P_{31} \vee P_{22}}$$

Soundness of Resolution?

Completeness of Resolution?

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
OK	OK B	P?	

Resolution Algorithm

□ To show $KB \models \alpha$, use proof by contradiction, i.e., show $KB \wedge \neg\alpha$ is unsatisfiable

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
          $\alpha$ , the query, a sentence in propositional logic

   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg\alpha$ 
   $new \leftarrow \{\}$ 
  loop do
    for each  $C_i, C_j$  in  $clauses$  do
       $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if  $resolvents$  contains the empty clause then return true
       $new \leftarrow new \cup resolvents$ 
    if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```

WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg \alpha$ is unsatisfiable

$$\neg P_{21} \vee B_{11}$$

$$\neg B_{11} \vee P_{12} \vee P_{21}$$

$$\neg P_{12} \vee B_{11}$$

$$\neg B_{11}$$

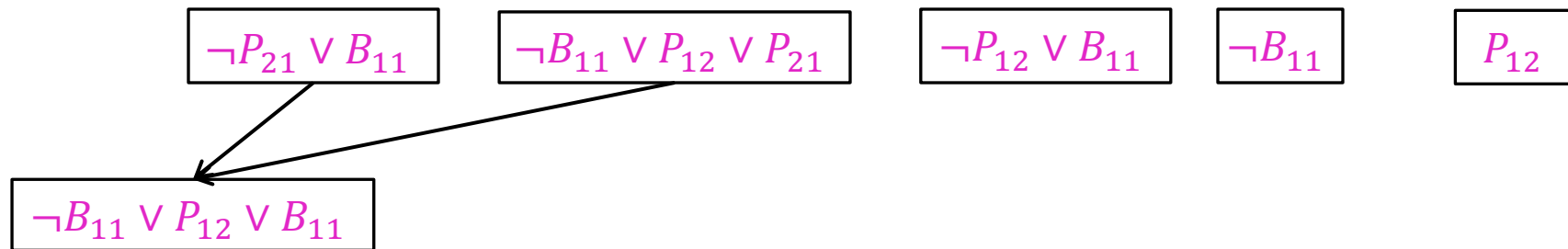
$$P_{12}$$

WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg \alpha$ is unsatisfiable

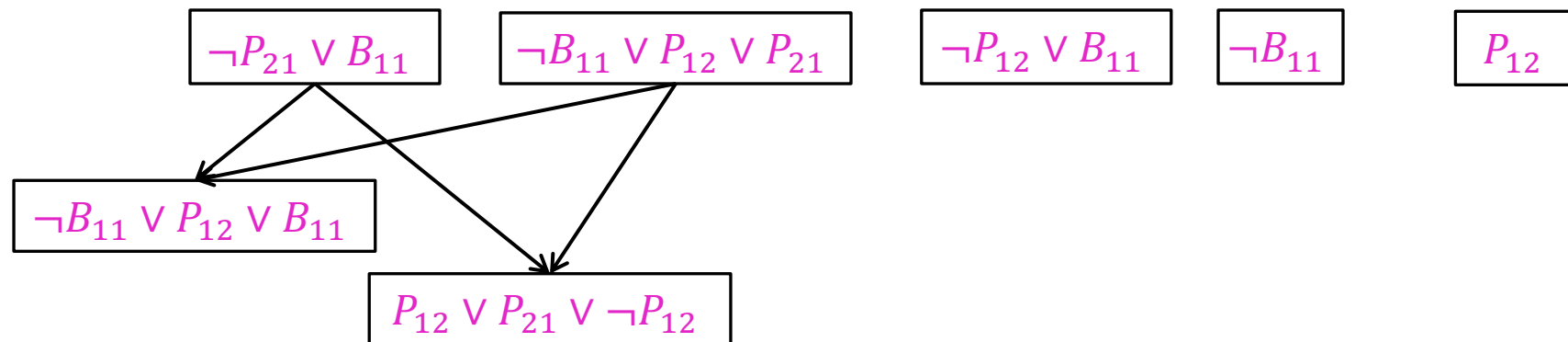


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable

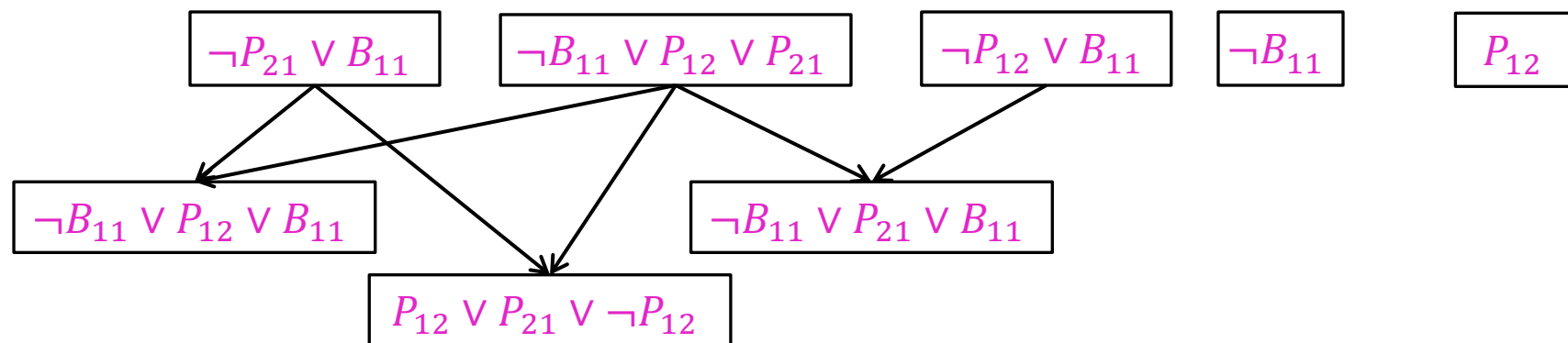


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable

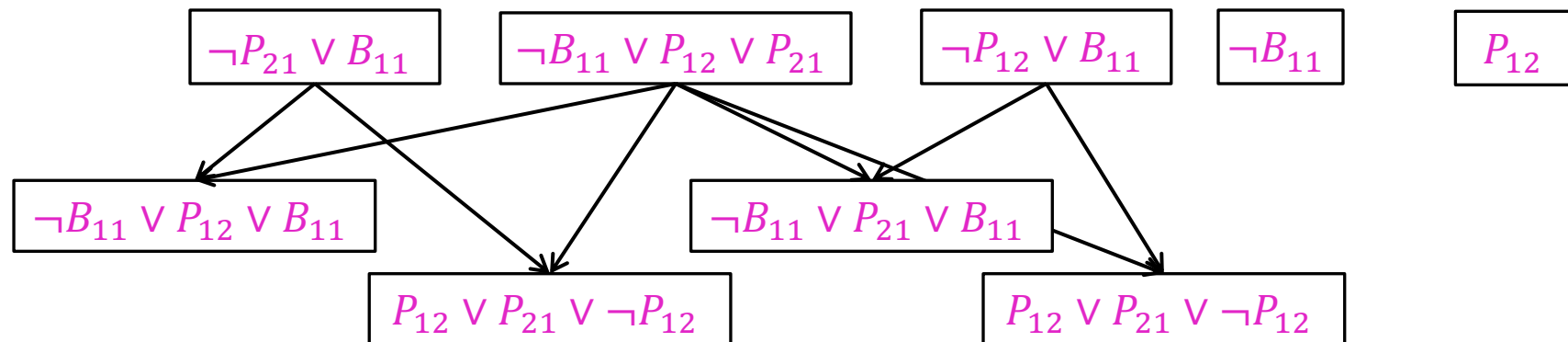


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable

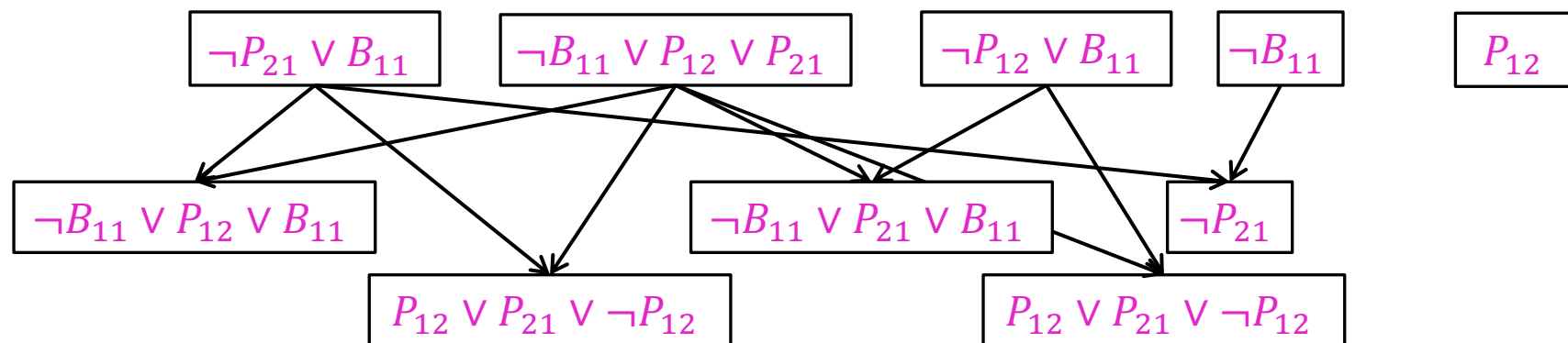


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable

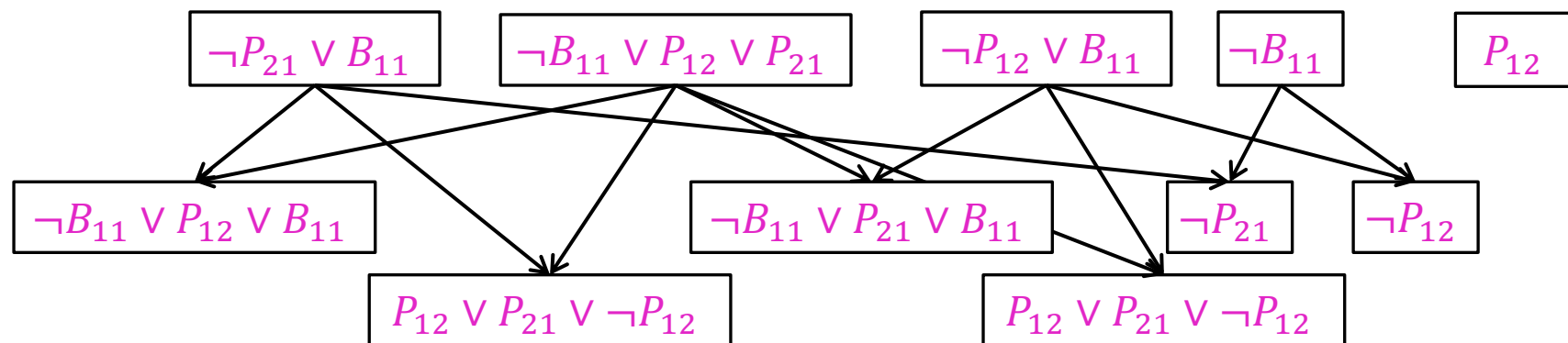


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable

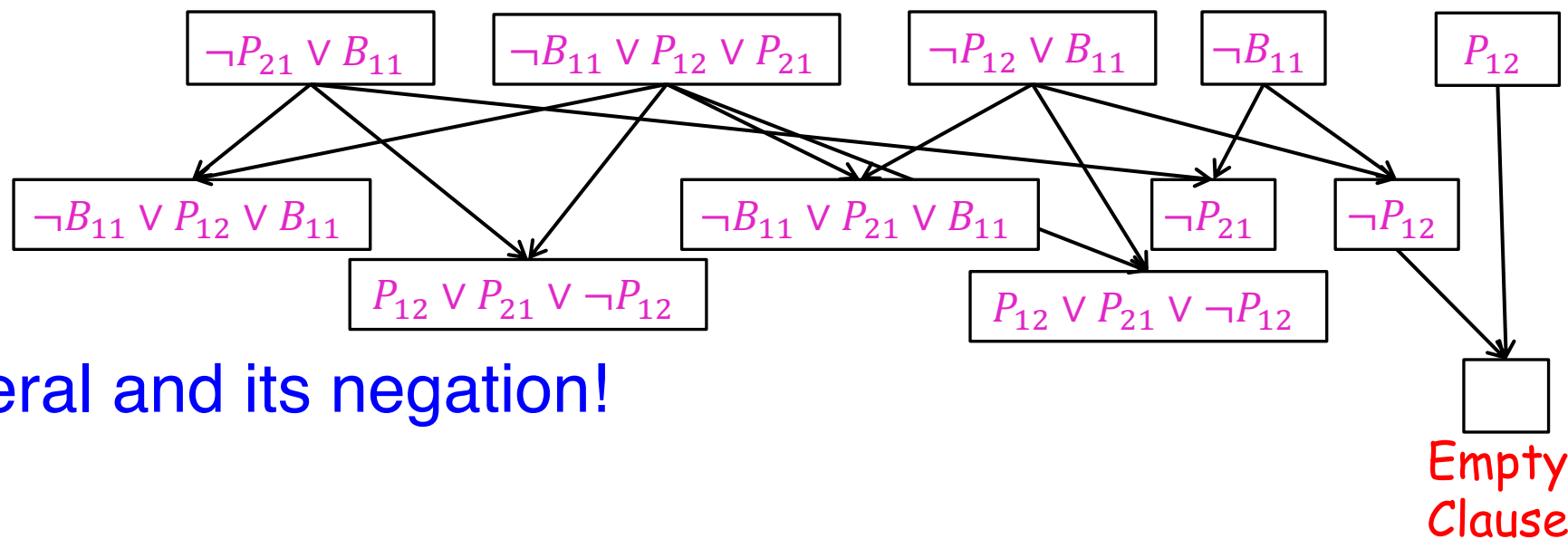


WW Example

□ Given $\neg B_{11}$, prove $\neg P_{12}$

$$KB = (B_{11} \Leftrightarrow (P_{12} \vee P_{21})) \wedge \neg B_{11}, \quad \alpha = \neg P_{12}$$

□ Resolution: Convert to CNF and show $KB \wedge \neg\alpha$ is unsatisfiable



□ A literal and its negation!

Resolution Algorithm (1)

□ To show $KB \models \alpha$, use proof by contradiction, i.e., show $KB \wedge \neg\alpha$ is unsatisfiable

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
          $\alpha$ , the query, a sentence in propositional logic

   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg\alpha$ 
   $new \leftarrow \{\}$ 
  loop do
    for each  $C_i, C_j$  in  $clauses$  do
       $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if  $resolvents$  contains the empty clause then return true
       $new \leftarrow new \cup resolvents$ 
    if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```


Resolution Algorithm (2)

□ Sound

- Application of resolution rule, which is sound

□ Complete

- Define $RC(S)$ - resolution closure – set of all clauses derivable by repeated application of the resolution rule to clauses in S or their derivatives
- $RC(S)$ must be finite – finite number of distinct clauses that can be constructed out of the symbols - $P_1, \dots, P_k$ that appear in S

□ *Ground Resolution Theorem*: If a set of clauses is *unsatisfiable*, then the resolution closure of those clauses contains the empty clause.

Resolution Algorithm (3)

- ❑ Contrapositive: If the closure $RC(S)$ does not contain the empty clause, then S is satisfiable.
- ❑ To show S is satisfiable, we need to come up with one model of S (for the symbols $P_1, \dots, P_k$).
- ❑ For $i = 1, \dots, k$
 - If a clause in $RC(S)$ contains the literal $\neg P_i$ and all its other literals are false under the assignment chosen for $P_1, \dots, P_{i-1}$, then assign false to P_i
 - Otherwise, assign true to P_i
- ❑ Does this construction result in a model of S ?

Resolution Algorithm (4)

□ Let us assume the opposite

- At some stage i in the sequence assigning symbol P_i causes some clause C to become false.
- It must be the case that all the other literals in C must have already have been falsified by assignments to $P_1, \dots, P_{i-1}$.
- Thus C must be *false* $\vee$ *false* $\vee \dots \vee P_i$ or *false* $\vee$ *false* $\vee \dots \vee \neg P_i$
- If just one of them is present in $RC(S)$, the construction process will assign appropriate value to P_i to make C true.
- So C can be falsified only if both these clauses are in $RC(S)$
- That would mean the resolvent of these two clauses to be false - $P_1, \dots, P_{i-1}$ - contradicts our assumption that that the first falsified clause appears at stage i

Inference Technique II – Forward/Backward Chaining

❑ Requirement: Sentence need to be in Horn Form

- KB = conjunction of Horn clauses
- Horn Clause = literal / “(conjunction of literal) $\Rightarrow$ literal” (i.e. clause with at most 1 positive literal; exactly one positive literal – definite clause)

e.g. $KB = C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$



❑ Horn Clauses are closed under resolution! Prove.

❑ “Modus Ponens” Rule:

$$\frac{\alpha_1, \dots, \alpha_n, \quad (\alpha_1 \wedge \dots \wedge \alpha_n) \Rightarrow \beta}{\beta}$$

❑ Very natural and linear time complexity in size of KB

Forward Chaining

❑ Idea: Fire any rule whose premises are satisfied in *KB*, add its conclusion to *KB*, until query *q* is found

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional Horn clauses
         q, the query, a proposition symbol
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known in KB

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)

  return false
```

Forward Chaining - Example

□ Query = “Is Q true?”

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

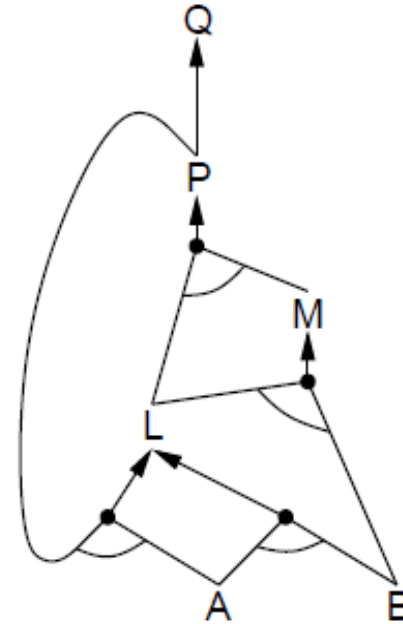
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

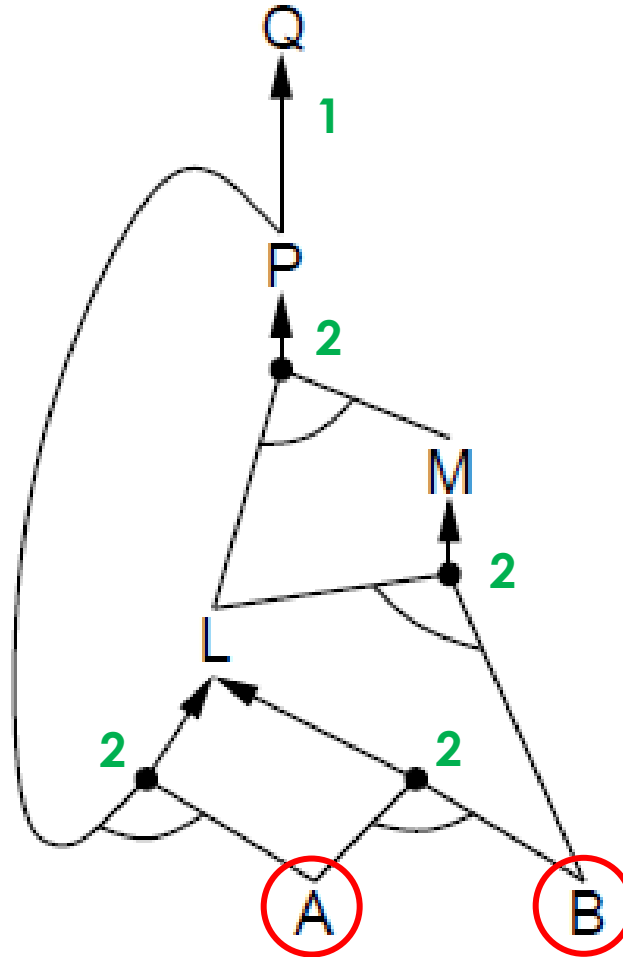
A

B

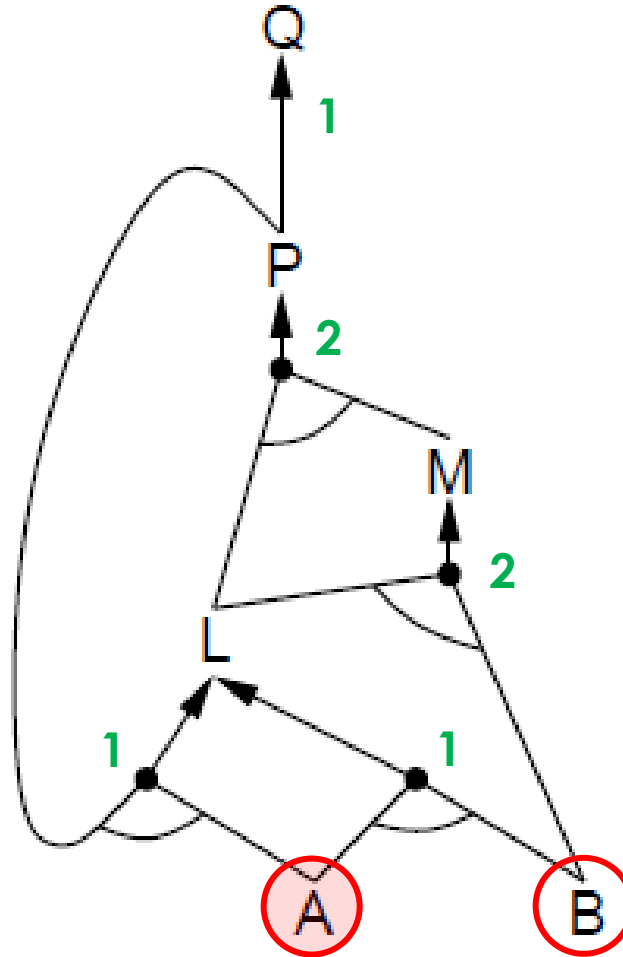


AND - OR Graph

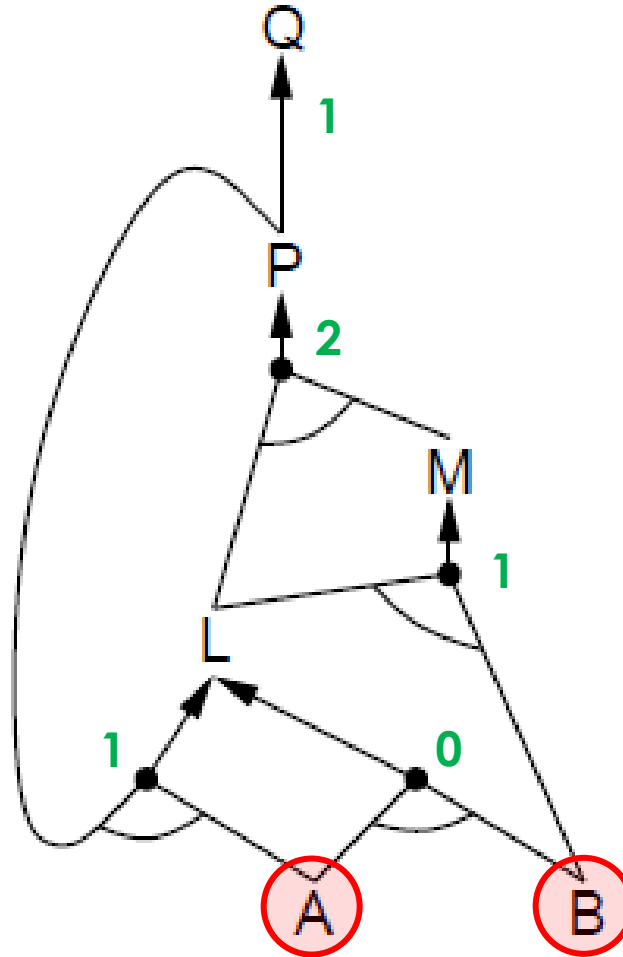
Forward Chaining Example



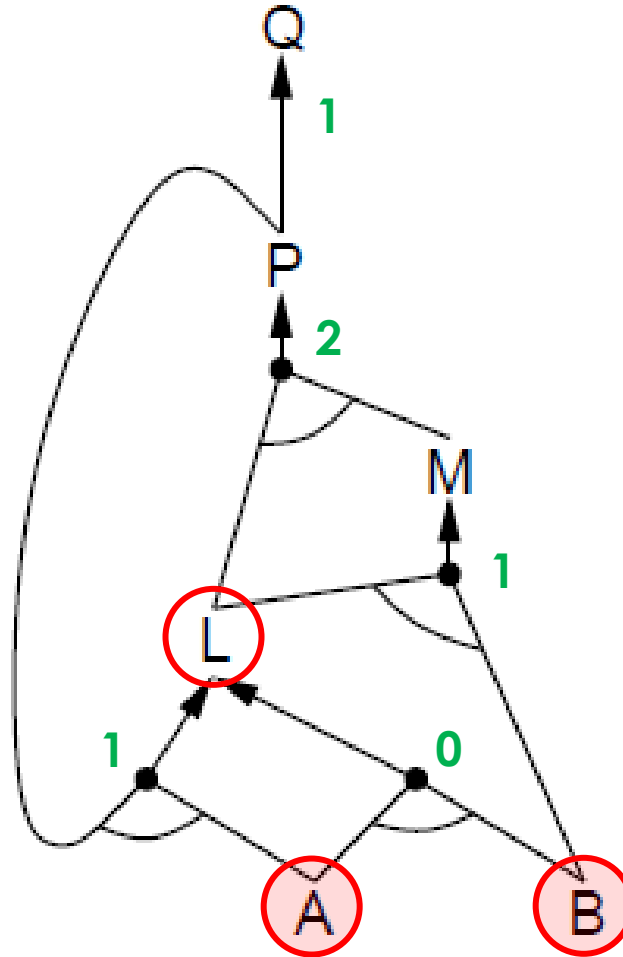
Forward Chaining Example



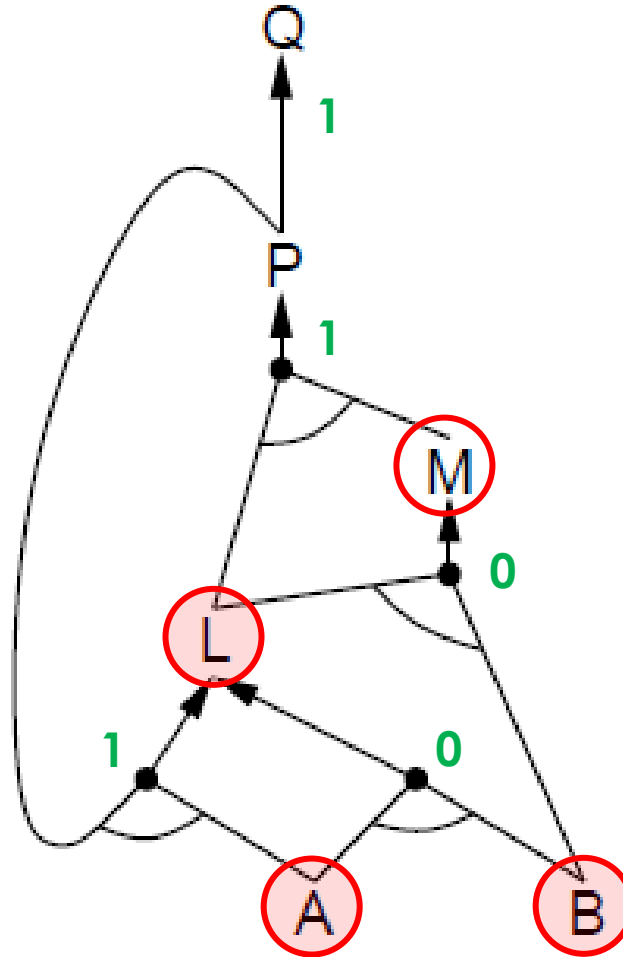
Forward Chaining Example



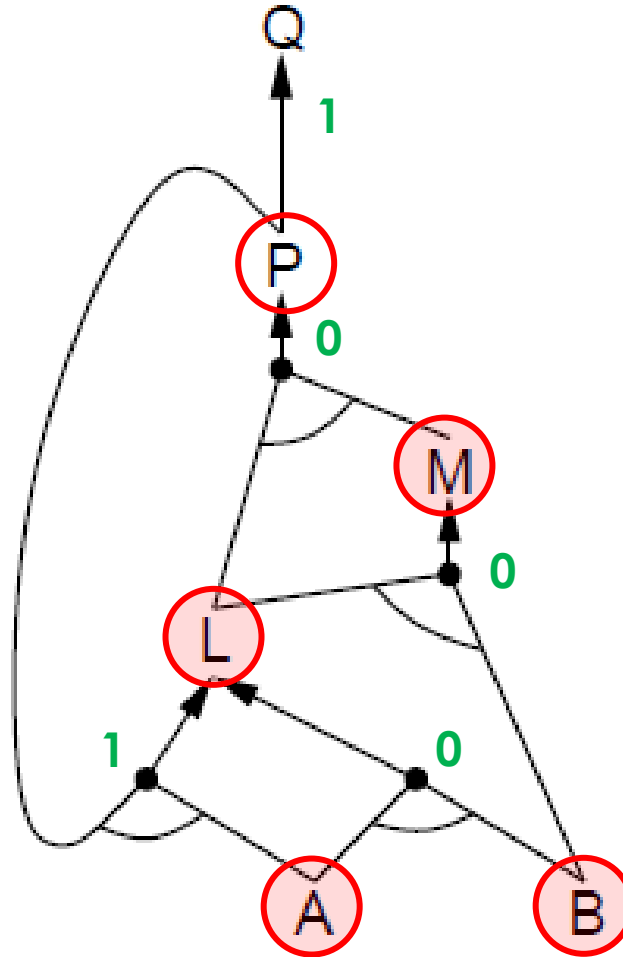
Forward Chaining Example



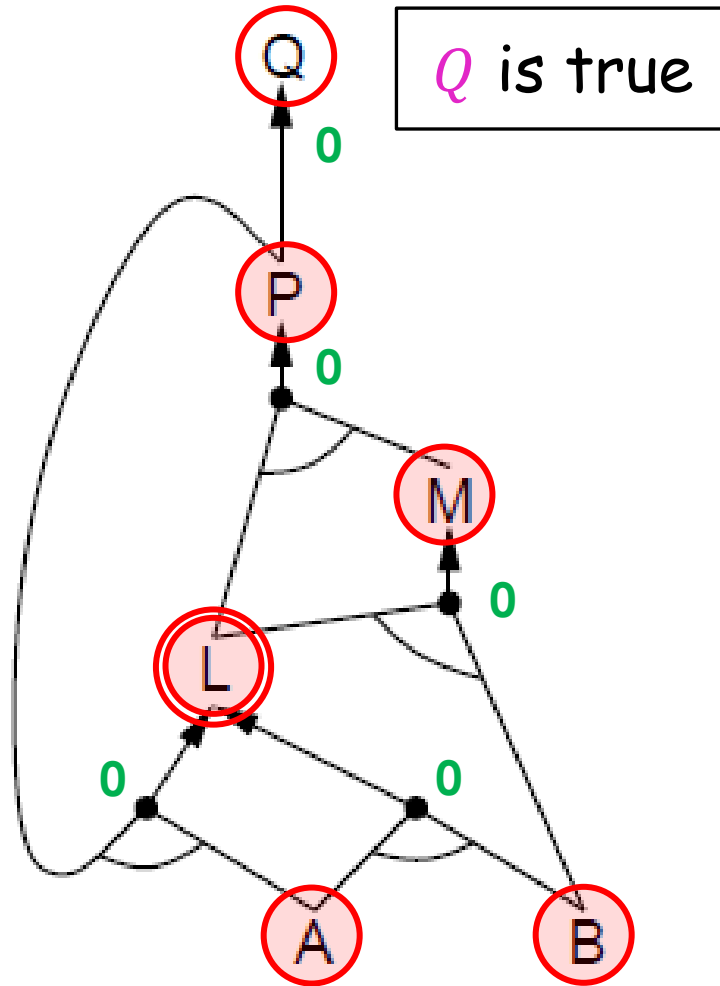
Forward Chaining Example



Forward Chaining Example



Forward Chaining Example



Forward Chaining

❑ Idea: Fire any rule whose premises are satisfied in *KB*, add its conclusion to *KB*, until query *q* is found

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional Horn clauses
         q, the query, a proposition symbol
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known in KB

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)
  return false
```

Sound and Complete

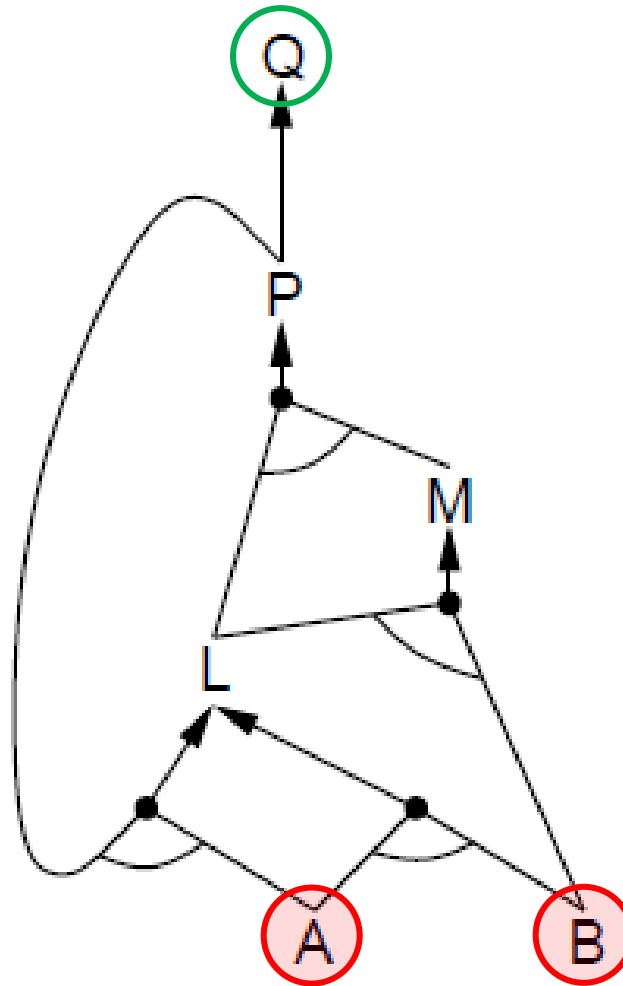
Forward Chaining - Completeness

- ❑ Completeness – every entailed atomic sentence will be derived
- ❑ Consider the **fixed point** of the algorithm (where no new inferences are possible)
- ❑ This will be a model for the KB
- ❑ Every definite clause in the KB is true in this model.

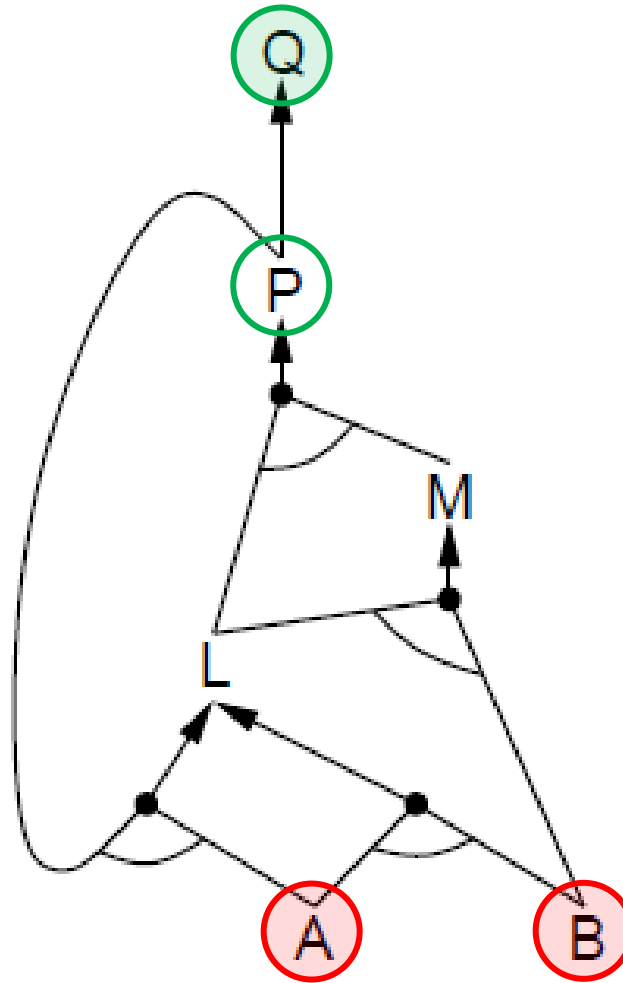
Backward Chaining

- ❑ Idea: work backwards from query q .
- ❑ To prove q
 - Check if q is known to be already true
 - Prove by backwards chaining all premises of some rule concluding q
 - (each premise to be proved is treated as a subgoal)
- ❑ Avoid Loops: check if new subgoal is already on goal stack
- ❑ Avoid Repeated Work: Check if new subgoal
 1. Has already been proved true
 2. Had already failed

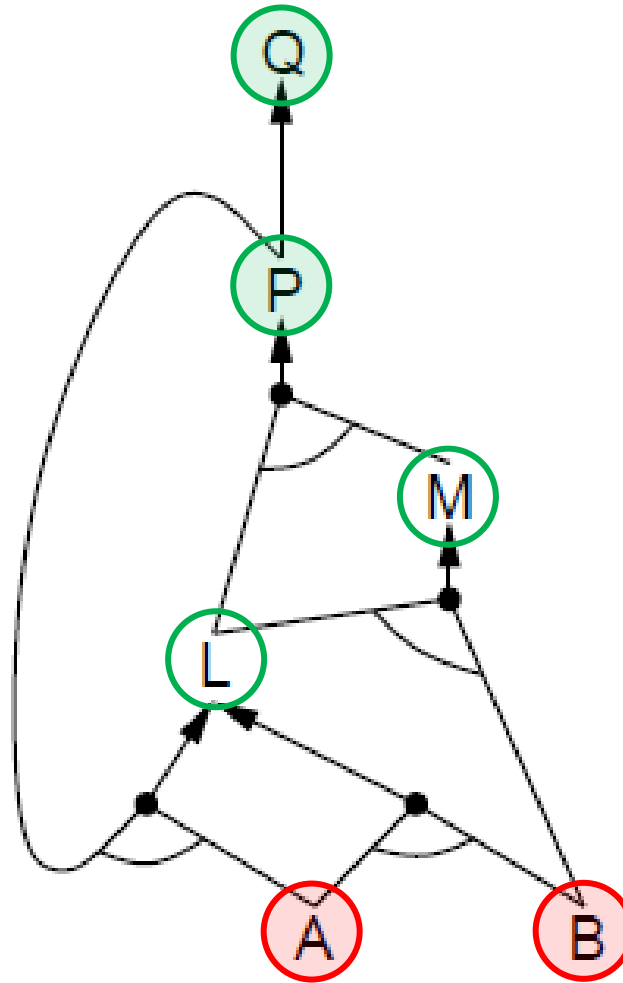
Backward Chaining Example



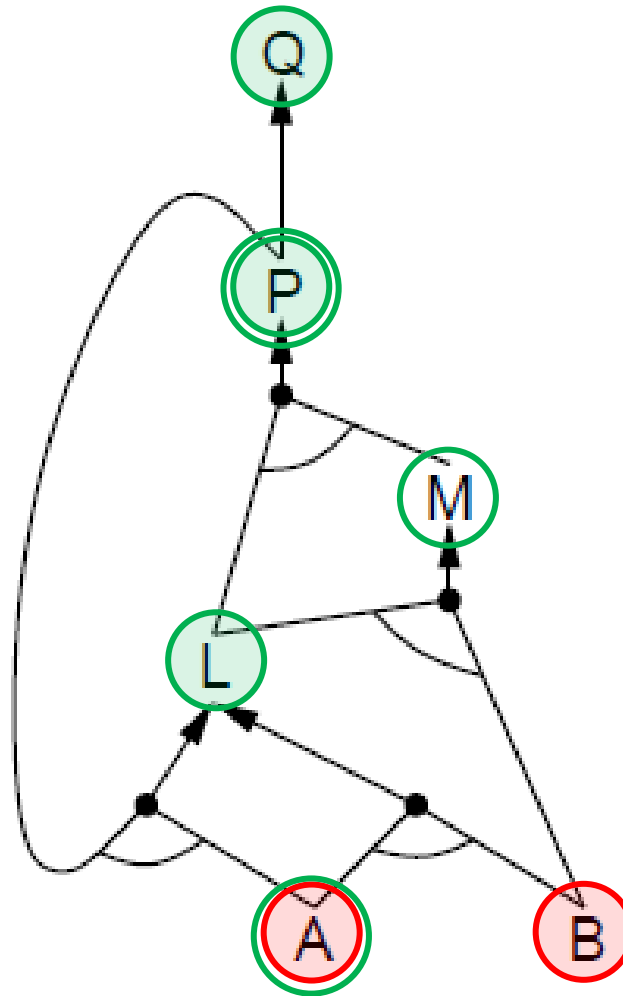
Backward Chaining Example



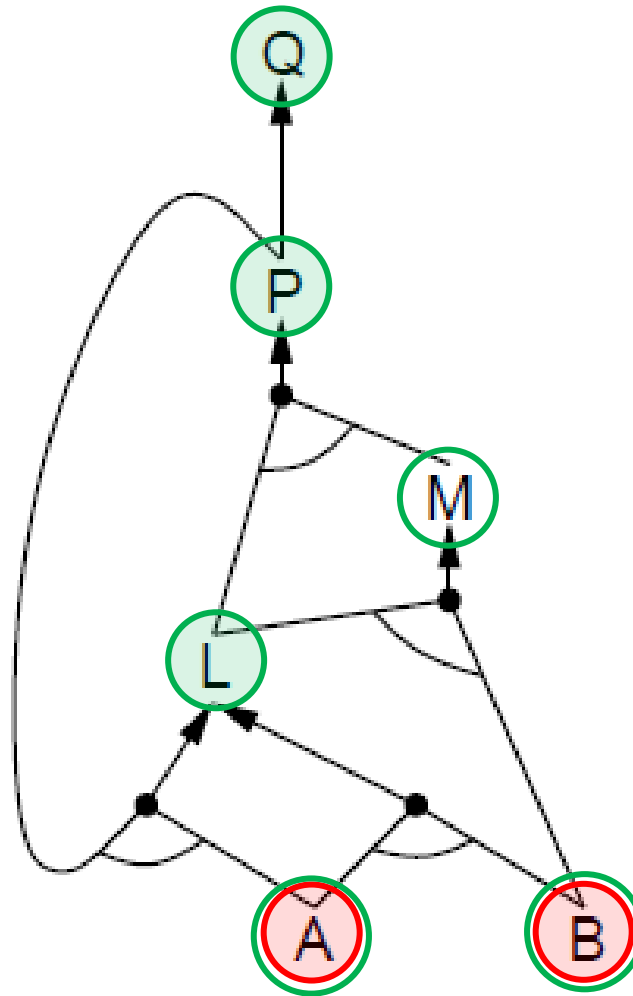
Backward Chaining Example



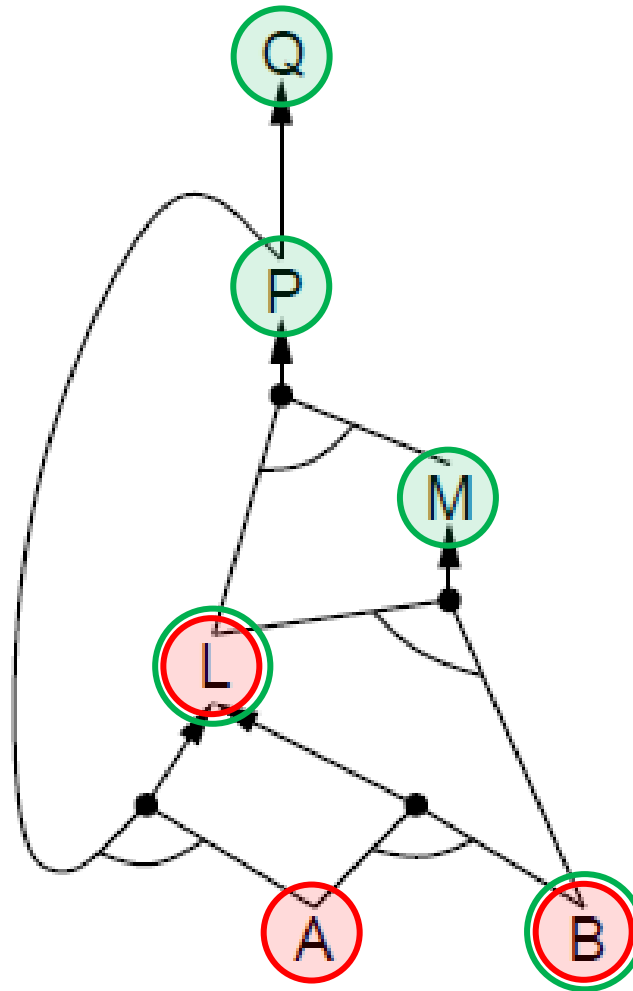
Backward Chaining Example



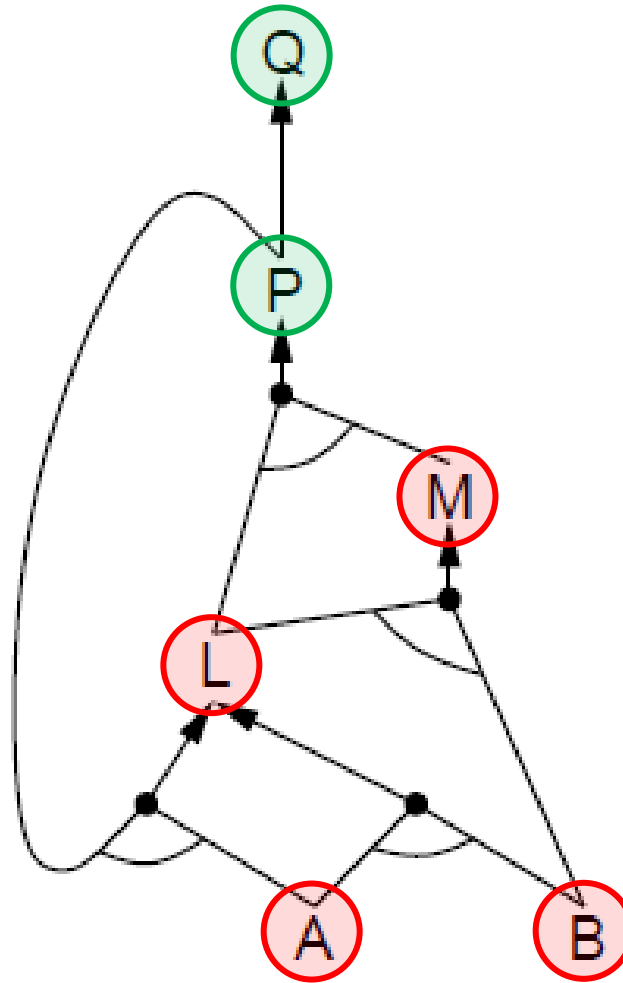
Backward Chaining Example



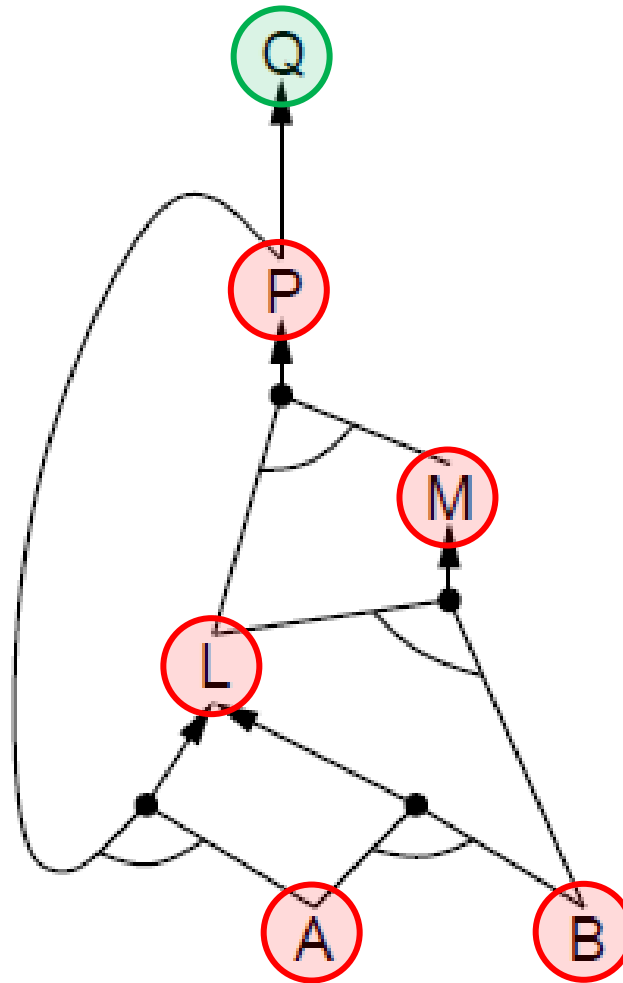
Backward Chaining Example



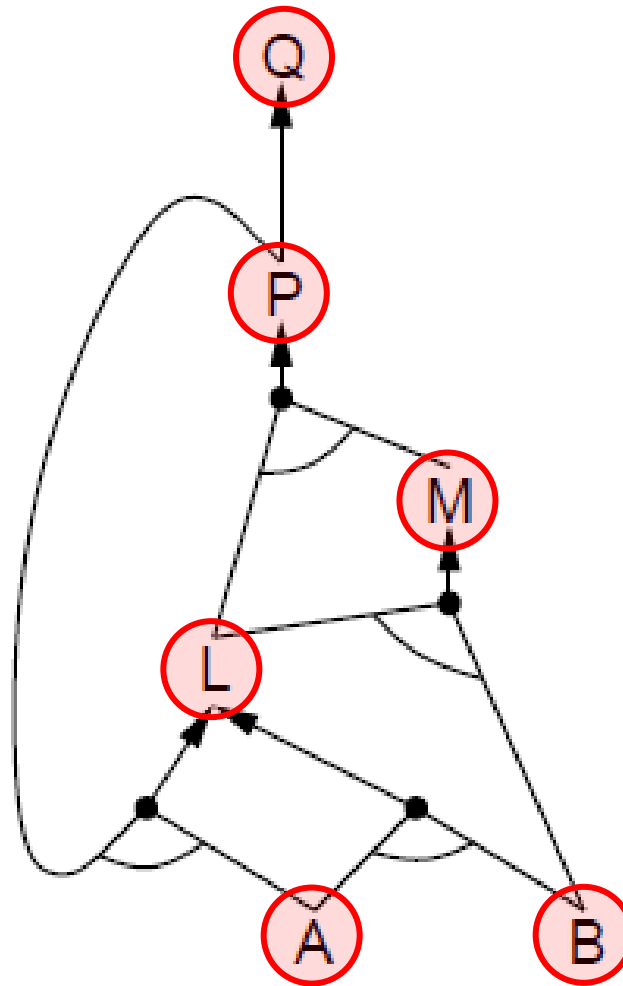
Backward Chaining Example



Backward Chaining Example



Backward Chaining Example



Forward vs Backward Chaining

- ❑ FC is 'data driven', unconscious processing
 - Object recognition, routine decisions
- ❑ FC may do lots of work that is irrelevant to the goal
- ❑ BC is 'goal driven', appropriate for problem solving
 - What shall I do now?
 - Where are my keys?
 - How do I get an S in this class?
- ❑ Complexity of BC *can be* much less than linear in size of *KB*
 - BC touches only relevant facts

Inference by Model Checking - Satisfiability

□ Entailment $KB \models \alpha$ is equivalent to showing $(KB \wedge \neg\alpha)$ is unsatisfiable.

□ SAT Problem

- Finding a model that satisfies all the clauses.

E.g. 2-CNF sentences (2 literals per clause)

$(\neg A \vee \neg B) \wedge (A \vee B) \wedge (A \vee \neg B)$ satisfiable?

$(\neg A \vee \neg B) \wedge (A \vee B) \wedge (A \vee \neg B) \wedge (\neg A \vee B)$ satisfiable?

□ SAT problem is NP Complete

□ Many combinatorial problems can be reduced to a SAT problem.

- A fast algorithm for SAT will yield fast algorithms other hard problems

The WalkSAT algorithm

❑ Local search algorithm

- Incomplete: may not always find a satisfying assignment even if one exists

❑ What should be the evaluation function?

- Number of satisfied clauses – maximize

❑ Balance between greediness and randomness

- Greediness - quickly approach a “satisfying” assignment
- Randomness - escape from local optima
- Each iteration
 - Randomly select a symbol for flipping (T-F or F-T)
 - OR select a symbol that maximizes the number of satisfied clauses

The WalkSAT Algorithm

function WALKSAT(*clauses*, *p*, *max_flips*) **returns** a satisfying model or *failure*

inputs: *clauses*, a set of clauses in propositional logic

p, the probability of choosing to do a “random walk” move, typically around 0.5

max_flips, number of flips allowed before giving up

model $\leftarrow$ a random assignment of *true/false* to the symbols in *clauses*

for *i* = 1 **to** *max_flips* **do**

if *model* satisfies *clauses* **then return** *model*

clause $\leftarrow$ a randomly selected clause from *clauses* that is false in *model*

with probability *p* flip the value in *model* of a randomly selected symbol from *clause*

else flip whichever symbol in *clause* maximizes the number of satisfied clauses

return *failure*

What happens when WalkSAT returns failure?

- Either clauses are unsatisfiable
- OR WalkSAT needs more time
 - If $\text{max_flips} = \infty$ and $p > 0$
 - WalkSAT will return a satisfying model (if one exists)
 - Else - loops into infinity

Landscape of SAT problems

□ $CNF_k(m, n)$

- k – number of literals per clause
- m – number of clauses in the sentence
- n – number of symbols in the sentence

□ $CNF_3(m, 50)$

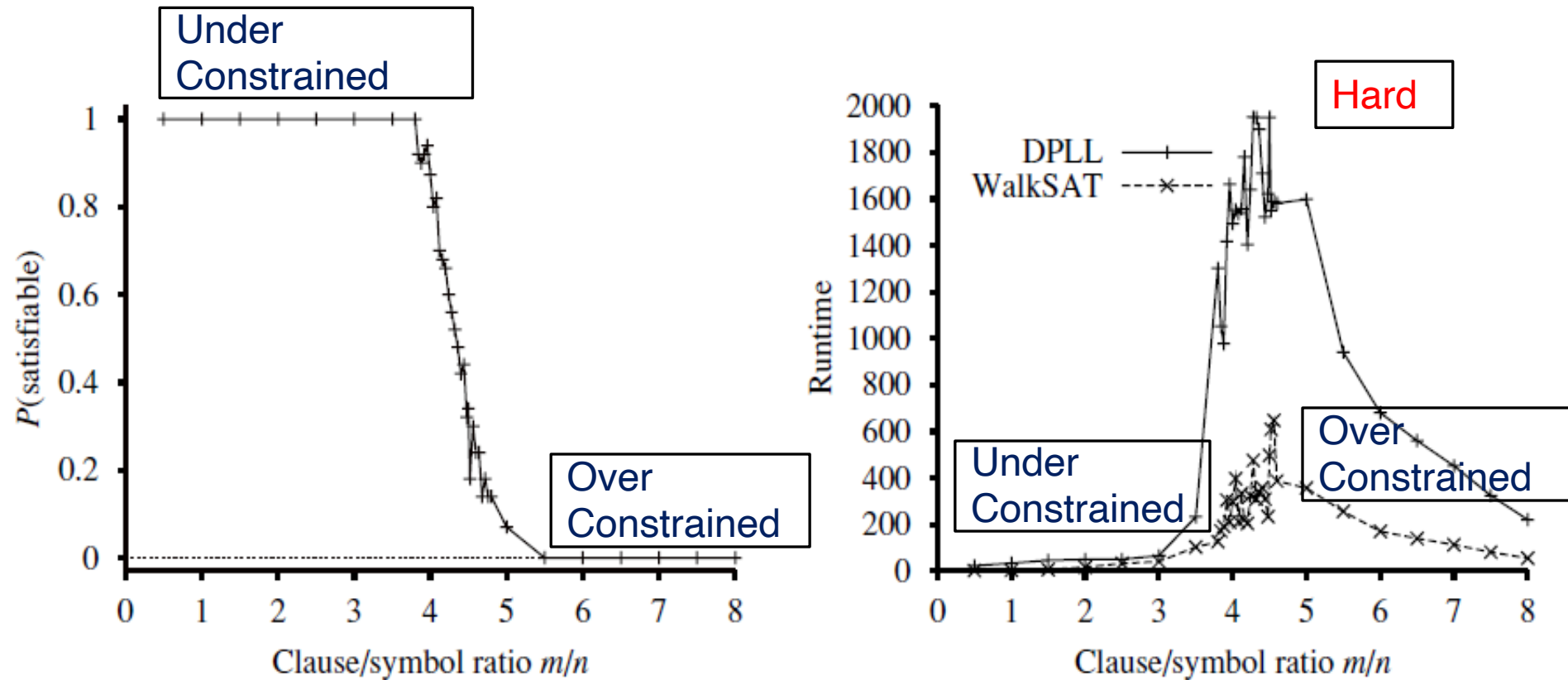
- Sentences with 3 literals per clause
- Overall there are 50 symbols

□ Clause to symbol ratio - m/n

□ Underconstrained problems - m/n - small

□ Overconstrained problems - m/n - large

Landscape of SAT problems



Propositional Logic - Limitations

❑ Does not scale to large size environments

❑ For the current WW

- 64 proposition symbols
- 155 sentences

❑ Lacks concise representations

❑ We would like to talk about objects and group of objects and relationships between them.

